Reproducing Figure 3 of LEXO: LLM-Assisted Program Regeneration for Supply-Chain Security

Abdelhak Chellal

May 2026

1 Introduction

This report documents my reproduction of Figure 3 from the paper *"Lexo: Eliminating Stealthy Supply-Chain Attacks via LLM-Assisted Program Regeneration"* (Lamprou et al., 2025).

Modern software depends on thousands of third-party packages. Attackers can inject malicious code that stays dormant until very specific conditions are met — for example, activating only on a specific platform, only when a certain environment variable is set, or only when connected to a live Bitcoin network. These are called *stealthy supply-chain attacks*. They evade standard testing because the malicious behavior never triggers during normal test runs.

LEXO's approach is simple: instead of trying to *detect* malicious code, it *regenerates* the package from scratch using only observable input/output behavior. Since malicious code causes side effects (network calls, file access) rather than changing return values, it is invisible to the regeneration process and does not appear in the new version.

Figure 3 of the paper evaluates regeneration correctness across 147 packages and 4 LLM models. I reproduced this experiment on 13 packages and 7 models, using simplifying assumptions where necessary.

2 Methodology

2.1 The Pipeline

I implemented the LEXO pipeline from scratch in Python, following the paper's description and the exact prompts from Appendix A. The pipeline has three stages:

1. **Input generation.** An LLM reads the package source code and generates test inputs. For example, for `is-number`, the LLM generates inputs like `[42]`, `["hello"]`, `[null]`, `[true]`.
2. **I/O pair collection.** Each input is run against the original package and the output is recorded — for example, `is-number(42) = true`, `is-number("hello") = false`. Malicious side effects are invisible here since only return values are captured.
3. **Regeneration.** The LLM infers a natural language algorithm from the I/O pairs, then generates new clean code. The original source is never consulted again, severing any connection to malicious code.

2.2 Modifications to the Paper's Prompts

The prompts from Appendix A were used as the base. Several additions were necessary in practice:

- **Strict JSON rules.** LLMs frequently return non-JSON values (`NaN`, `undefined`, Python-style `True/False`) or add comments inside JSON. A `STRICT RULES` block was added explicitly forbidding these.
- **Input format change.** The paper uses JavaScript expression syntax (e.g. `[x => x + 1]`) which Python cannot parse. We use JSON arrays of arrays (`[[1], [0], [-1]]`). The goal is identical — the LLM suggests inputs, the system runs them to get real outputs.
- **Function argument serialization.** `concat-map` and `just-filter-object` take functions as arguments, which are not JSON-serializable. These are serialized as strings and `eval()`'d at runtime: `[[1,2,3], "function(x){ return x*2; }"]`.
- **Retry with stricter reminder.** On failure, the model is retried up to 3 times with a stronger JSON format reminder and lower temperature (0.3 vs 0.7). The paper uses a coverage-guided revision loop; we use a simpler JSON-parse-failure trigger.

2.3 Models

The paper uses GPT-5 mini, GPT-4o, GPT-3.5, and Mistral 7B. I made the following changes:

- **GPT-5 mini** was substituted with **GPT-5.4 mini** — the original ID (`openai/gpt-5-mini`) returned empty responses via OpenRouter.
- **GPT-4o** was substituted with **GPT-4o mini** due to cost.
- Three models were added beyond the paper's set: **Claude 3.5 Haiku**, **DeepSeek V4 Flash**, and **Owl Alpha**, to extend the comparison.

All models were accessed via OpenRouter at a total cost under $1 for all experiments.

2.4 Packages

I evaluated 13 of the 15 packages from Table 2 of the paper. Two were dropped: `fast_blank` (Ruby C extension — bundler version conflicts) and `character-count` (C++ NAN library incompatible with Node.js v20). A third, `split-on-first`, was dropped due to ESM/CommonJS incompatibility. These represent infrastructure limitations unrelated to the LEXO pipeline itself.

For `primality` (Python), I regenerated 6 of 7 functions. The 7th (`rand_prime`) was added as a simple wrapper around `between()` with `random.choice()`.

3 Results

Model	Paper equiv.	At 100%	Avg score
GPT-5.4 mini	GPT-5 mini	9/13	80.1%
Owl Alpha	—	4/13	52.0%
DeepSeek V4 Flash	—	5/13	49.5%
GPT-4o mini	GPT-4o	2/13	41.3%
Claude 3.5 Haiku	—	3/13	38.7%
GPT-3.5 Turbo	GPT-3.5	1/13	32.2%
Mistral 7B	Mistral 7B	0/13	11.9%

Table 1: Final results (3 retries, 13 packages). "At 100%" = passed all developer tests.

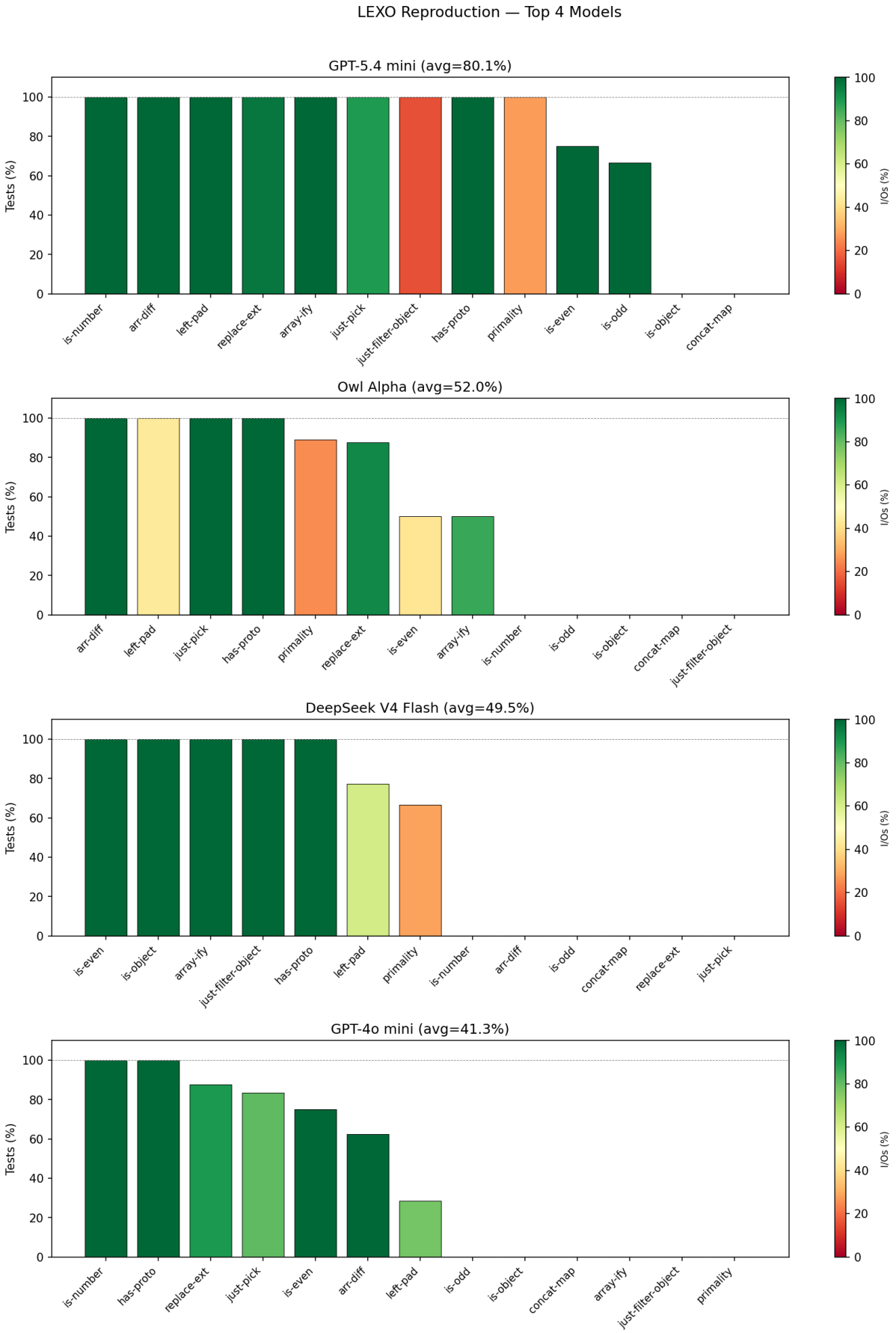

Figure 1: Top 4 models by average score. Each bar = one package. Bar height = % of developer tests passed. Color = % of I/O pairs matched (dark green = high, red = low).

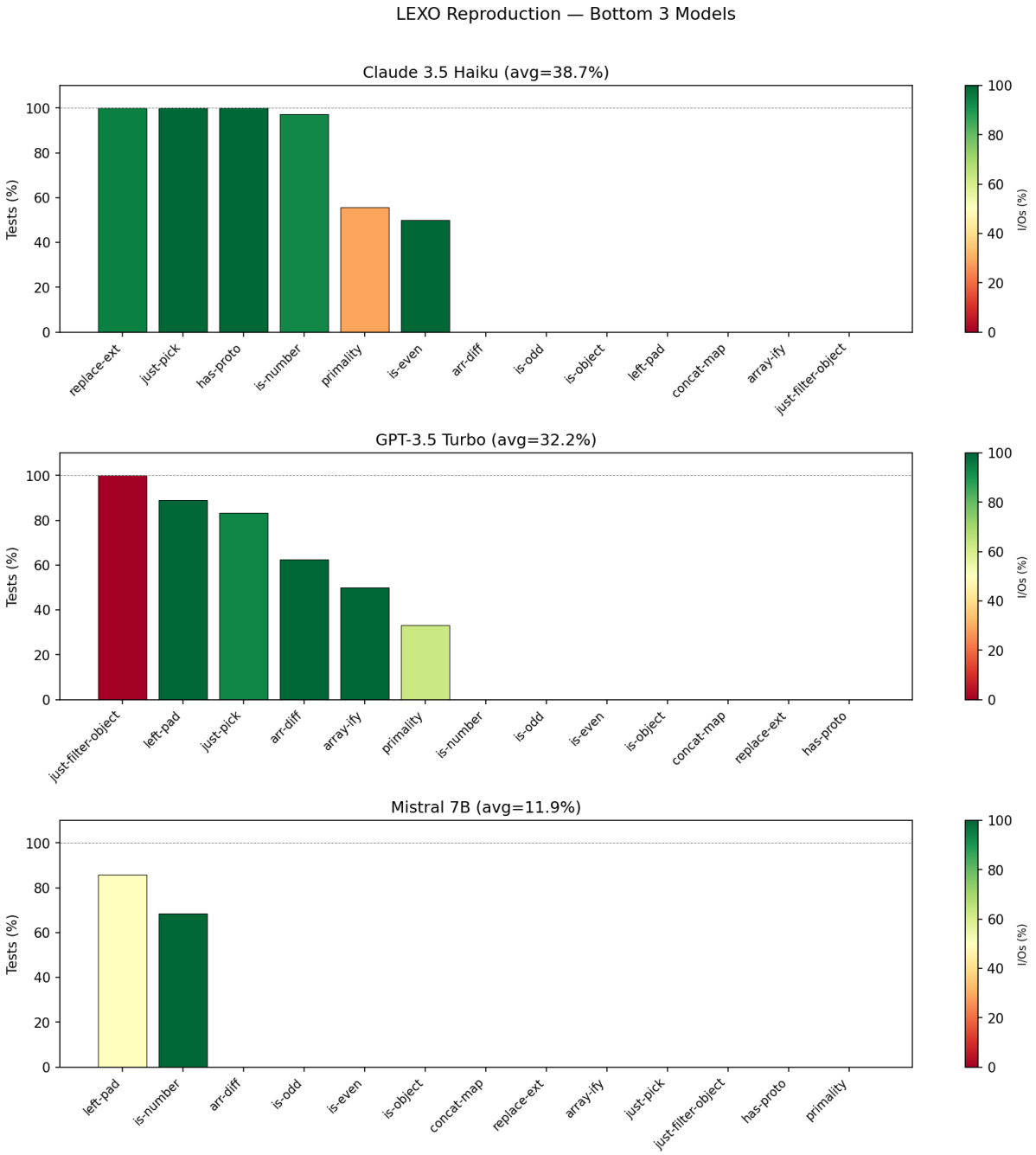

Figure 2: Bottom 3 models. Same axes.

3.1 Comparison with the Paper

Model	Paper (147 pkgs, at 100%)	Ours (13 pkgs, at 100%)
GPT-5 mini / GPT-5.4 mini	86/147 (59%)	9/13 (69%)
GPT-4o / GPT-4o mini	59/147 (40%)	2/13 (15%)
GPT-3.5 Turbo	35/147 (24%)	1/13 (8%)
Mistral 7B	3/147 (2%)	0/13 (0%)

Table 2: Comparison with paper results.

The ordering of models matches the paper exactly: GPT-5 mini > GPT-4o > GPT-3.5 > Mistral 7B. Our percentages for GPT-4o mini and GPT-3.5 are lower than the paper's, which is expected — the paper implements a full coverage-guided revision loop that we did not replicate, and evaluates more packages reducing the variance.

4 Observations

The idea is simple and powerful. LEXO does not need to understand malicious code at all. The key insight — that malicious behavior is side-effectful and therefore invisible in return values — makes the approach elegant and robust. I find it a great example of solving a hard problem by reframing it.

LLM non-determinism is a real challenge. The same prompt, same package, and same model can succeed on one run and fail with a JSON parse error on the next. Building a reliable pipeline on top of non-deterministic models requires significant engineering effort — retry logic, output normalization, and format enforcement. The paper acknowledges this with its revision loop, but even so, perfect reliability is not guaranteed.

Model quality is the bottleneck. Most failures were not code generation failures — they were input generation failures. Weaker models (Mistral, GPT-3.5) could not reliably produce valid JSON even after 3 retries. This points to an interesting research direction: fine-tuning small open-source models specifically on structured JSON output tasks could make LEXO viable without expensive frontier models.

The cost is negligible. All experiments — multiple runs across 7 models and 13 packages — cost under $1. This makes LEXO practical for real CI/CD pipelines.

Language-agnostic in theory, engineering-intensive in practice. The paper claims LEXO is language-agnostic, which is conceptually true. In practice, each language requires custom serialization, test framework parsing, and runtime integration. Ruby and C++ packages could not be included due to native compilation failures, and Python required a separate sub-pipeline.

5 Conclusion

This reproduction confirms the paper's core finding: stronger LLMs produce better regenerations, and the approach is practical and cheap. The model ordering in our results matches the paper exactly.

The main gap is the absence of a coverage-guided revision loop, which the paper shows is important for correctness. Implementing it fully, and exploring whether fine-tuned smaller models could replace frontier models for this task, would be interesting directions for future work.

Repository: `https://github.com/Abdelhak-Chellal/Lexo-repro`
Total API cost: $< \$1$ across all experiments.